One problem that remains is that `RegisterUser` is still tied to an *implementation* rather than an *abstraction*. Coding to abstractions (often interfaces) is a common design pattern that helps further reduce the coupling of a system, as you're not tied to a single implementation. This pattern is particularly useful for making classes testable, as you can create stub or mock implementations of your dependencies for testing purposes, as shown in figure 8.3.

TIP You can choose among many mocking frameworks. I'm most familiar with Moq, but NSubstitute and FakeItEasy are also popular options.

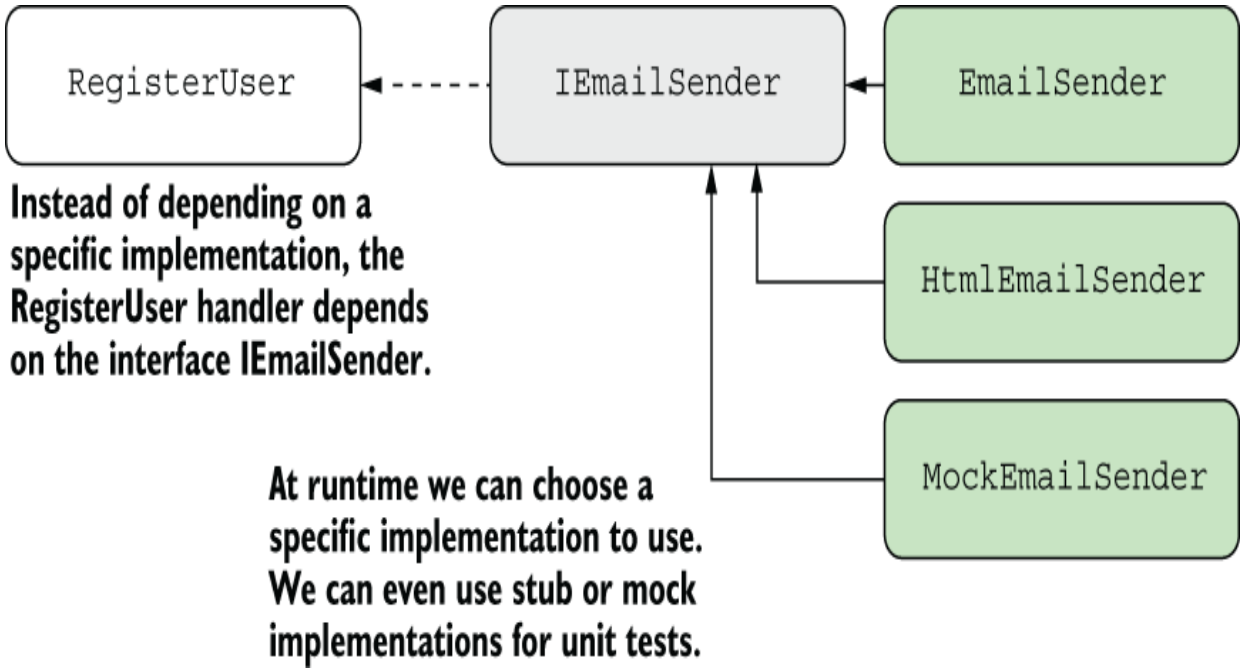


Figure 8.3 By coding to interfaces instead of an explicit implementation, you can use different `IEmailSender` implementations in different scenarios, such as a `MockEmailSender` in unit tests.

As an example, you might create an `IEmailSender` interface, which `EmailSender` would implement: